

MASTER CODEBASE ARCHITECTURE REPORT: InterviewAI

SECTION 0 — Codebase Inventory

Folder Inventory

- **lib/**: Contains core code entry (**main.dart**), configurations (**firebase_options.dart**), global patterns (**core/**), UI routes (**features/**), and common widgets (**shared/**). (verified)
- **lib/core/**: Houses global utilities, model declarations, Riverpod providers, service wrappers, and styles. (verified)
- **lib/core/constants/**: Holds constant variables like sizing metrics. (verified)
- **lib/core/models/**: Serialization schemas for categories, questions, responses, feedback, and user files. (verified)
- **lib/core/providers/**: Riverpod data flow hooks declarations. (verified)
- **lib/core/services/**: Class wrappers executing communications with Firebase and Gemini endpoints. (verified)
- **lib/core/theme/**: Theme definitions, style sheets, and typography profiles. (verified)
- **lib/features/**: Feature modules divided by route contexts (e.g. auth, dashboard, profile, home splash). (verified)
- **lib/shared/**: Reusable shared UI widgets. (verified)

File Inventory

- **lib/main.dart**: Entrypoint of the app. Initializes dotenv and Firebase options, configures GoRouter paths, and wraps the widget hierarchy with **ProviderScope**. (verified)
- **lib/firebase_options.dart**: Generated options mapping credentials to regional platforms. (verified)
- **lib/core/constants/constants.dart**: Defines padding, sizing, alignment constants, and max widths. (verified)
- **lib/core/theme/colors.dart**: Harmonious visual palette maps. (verified)
- **lib/core/theme/typography.dart**: Maps Google Fonts to Material text styles. (verified)
- **lib/core/theme/theme.dart**: Light theme visual configurations. (verified)
- **lib/core/models/category_model.dart**: **Category** structure. (verified)
- **lib/core/models/question_model.dart**: **Question** structure. (verified)
- **lib/core/models/feedback_model.dart**: **InterviewFeedback** structure. (verified)
- **lib/core/models/session_model.dart**: **SessionSummary**, **SessionDetail**, and **AnswerDetail** structures. (verified)
- **lib/core/models/user_model.dart**: Unused **User** structure. (verified)
- **lib/core/providers/api_providers.dart**: Rivers of state managers. (verified)
- **lib/core/services/auth_service.dart**: Interacts with Firebase Auth for user management. (verified)
- **lib/core/services/profile_service.dart**: Modifies profile metadata maps. (verified)
- **lib/core/services/gemini_service.dart**: Generates and checks responses with AI prompts. (verified)
- **lib/core/services/history_service.dart**: Manages Firestore writes and transactions. (verified)
- **lib/core/services/interview_service.dart**: Unused legacy FastAPI wrapper. (verified)
- **lib/core/api/api_client.dart**: Unused legacy REST client. (verified)
- **lib/features/home/splash_screen.dart**: Redirect trigger for active auth sessions. (verified)
- **lib/features/home/onboarding_screens.dart**: Static slides describing app features. (verified)
- **lib/features/auth/login_screen.dart**: User login interface. (verified)
- **lib/features/auth/register_screen.dart**: User signup interface. (verified)
- **lib/features/auth/forgot_password_screen.dart**: Reset password interface. (verified)
- **lib/features/dashboard/dashboard_screen.dart**: Dashboard home listing user progress statistics. (verified)
- **lib/features/dashboard/category_screen.dart**: Selection grid of practice categories. (verified)
- **lib/features/dashboard/practice_screen.dart**: Interactive mock practice session. (verified)
- **lib/features/dashboard/feedback_screen.dart**: Score indicator and AI recommendations page. (verified)
- **lib/features/dashboard/history_screen.dart**: History log list of past sessions. (verified)
- **lib/features/dashboard/session_details_screen.dart**: Historical detail card. (verified)
- **lib/features/profile/profile_screen.dart**: Display and update name settings. (verified)

Screen Inventory

- **SplashScreen** ([**splash_screen.dart**]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/home/splash_screen.dart)): Performs startup checks.

- **OnboardingScreens** ([onboarding_screens.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/home/onboarding_screens.dart)): Displays application highlights.
- **LoginScreen** ([login_screen.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/login_screen.dart)): Captures and validates user credentials.
- **RegisterScreen** ([register_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/register_screen.dart)): Handles email creation and user profile registration.
- **ForgotPasswordScreen** ([forgot_password_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/forgot_password_screen.dart)): Sends password recovery emails.
- **DashboardScreen** ([dashboard_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/dashboard_screen.dart)): Core dashboard.
- **CategoryScreen** ([category_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/category_screen.dart)): Domain category selector.
- **PracticeScreen** ([practice_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/practice_screen.dart)): Direct answer prompt screen.
- **FeedbackScreen** ([feedback_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/feedback_screen.dart)): Post-practice feedback dashboard.
- **HistoryScreen** ([history_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/history_screen.dart)): Practice history logs list.
- **SessionDetailsScreen** ([session_details_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/session_details_screen.dart)): Historical breakdown card.
- **ProfileScreen** ([profile_screen.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/profile/profile_screen.dart)): Name settings updates.

Provider Inventory

All declared inside [api_providers.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/providers/api_providers.dart):

- **authServiceProvider**: Provides **AuthService** interface.
- **geminiServiceProvider**: Provides **GeminiService** interface.
- **historyServiceProvider**: Provides **HistoryService** interface.
- **profileServiceProvider**: Provides **ProfileService** interface.
- **categoriesProvider**: Async categories list query loader.
- **historyProvider**: Session Summaries async list query loader.
- **sessionDetailProvider**: Detail data family provider loader.
- **profileProvider**: Profile settings map loader.
- **geminiQuestionProvider**: Aynchronous generative question family loader.

Service Inventory

- **AuthService** ([auth_service.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/auth_service.dart)): Accesses Firebase Authentication APIs.
- **ProfileService** ([profile_service.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/profile_service.dart)): Reads and writes user profiles in Firestore.
- **GeminiService** ([gemini_service.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/gemini_service.dart)): Accesses the Gemini API.
- **HistoryService** ([history_service.dart]
(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/history_service.dart)): Manages transaction scripts for database session records.

Model Inventory

- **Category** ([category_model.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/models/category_model.dart)): Serializes domain categories.
- **Question** ([question_model.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/models/question_model.dart)): Serializes questions.
- **InterviewFeedback** ([feedback_model.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/models/feedback_model.dart)): Serializes response feedback.
- **SessionSummary** ([session_model.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/models/session_model.dart)): Lightweight summary profile of completed sessions.
- **SessionDetail** ([session_model.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/models/session_model.dart)): Deep structure holding details of a completed practice session.
- **AnswerDetail** ([session_model.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/models/session_model.dart)): Mapped item detailing response text, question text, scores, strengths, weaknesses, and model answers.

Firebase Usage Inventory

- **Firestore Auth**: Session checks, authentication validation, signup operations, password reset execution, and signouts (verified in [auth_service.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/auth_service.dart)).
- **Cloud Firestore**: Profiles, categories, and session record configurations (verified in [profile_service.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/profile_service.dart), [history_service.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/history_service.dart), and [api_providers.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/providers/api_providers.dart)).

AI Usage Inventory

- **Question Generation**: Creates category-tailored interview questions in real time using **gemini-1.5-flash** (verified in [gemini_service.dart:L16](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/gemini_service.dart#L16)).
- **Answer Evaluation**: Scores answers and generates feedback lists using a JSON structure parsed through **jsonDecode** (verified in [gemini_service.dart:L32](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/gemini_service.dart#L32)).

SECTION 1 — Executive Summary

- **Project Name**: InterviewAI (verified from [pubspec.yaml](file:///c:/Users/mh978/Downloads/interviewai%20(2)/pubspec.yaml#L1))
- **Purpose**: A mobile application to help users practice mock interviews by generating industry-specific questions, scoring their answers, and providing structured feedback (verified in [practice_screen.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/practice_screen.dart#L49-L93) and [history_service.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/history_service.dart#L7-L54)).
- **Main Features**:
 1. Signup, login, password recovery, and session persistence using Firebase Auth.
 2. Live category-specific question generation and answer evaluation via the Gemini Generative Model API.
 3. Interactive feedback reporting (overall score, strengths, weaknesses, model answer) parsed from Gemini's JSON payloads.
 4. Practice session history with atomic updates to user stats (sessions counter and average score) in a Firestore transaction.
- **Target Users**: Job seekers looking for tailored domain-specific interview practice.
- **Technology Stack**: Dart, Flutter, Riverpod, GoRouter, Firebase Core, Firebase Auth, Cloud Firestore, Google Generative AI (verified from dependencies in [pubspec.yaml](file:///c:/Users/mh978/Downloads/interviewai%20(2)/pubspec.yaml)).
- **High-Level Architecture**: decoupled Repository/Service pattern mapped onto Riverpod Providers, ensuring isolation between the visual layer (Screens), logical triggers (Providers), core client layers (Services), and remote data engines (Firebase Auth, Cloud Firestore, and the Gemini API).

SECTION 2 — Project Structure

```

lib/
├─ main.dart
├─ firebase_options.dart

```

```

├── core/
│   ├── constants/
│   │   └── constants.dart
│   ├── models/
│   │   ├── category_model.dart
│   │   ├── feedback_model.dart
│   │   ├── question_model.dart
│   │   ├── session_model.dart
│   │   └── user_model.dart
│   ├── providers/
│   │   └── api_providers.dart
│   ├── services/
│   │   ├── auth_service.dart
│   │   ├── gemini_service.dart
│   │   ├── history_service.dart
│   │   └── profile_service.dart
│   └── theme/
│       ├── colors.dart
│       ├── theme.dart
│       └── typography.dart
├── features/
│   ├── auth/
│   │   ├── forgot_password_screen.dart
│   │   ├── login_screen.dart
│   │   └── register_screen.dart
│   ├── dashboard/
│   │   ├── category_screen.dart
│   │   ├── dashboard_screen.dart
│   │   ├── feedback_screen.dart
│   │   ├── history_screen.dart
│   │   ├── practice_screen.dart
│   │   └── session_details_screen.dart
│   ├── home/
│   │   ├── onboarding_screens.dart
│   │   └── splash_screen.dart
│   └── profile/
│       └── profile_screen.dart
└── shared/
    ├── widgets/
    │   ├── custom_button.dart
    │   ├── custom_card.dart
    │   └── buttons/
    │       ├── elevated_button.dart
    │       ├── outlined_button.dart
    │       └── text_button.dart

```

Folder Responsibilities & Decoupling Layering

- **UI Widgets Layer** (**lib/features/** & **lib/shared/**): Declarative components that watch Riverpod providers to render state variations (data, loading, error).
- **Logical Provider Layer** (**lib/core/providers/**): State management layer. Listens to service method changes and updates the UI layer.
- **Business Service Layer** (**lib/core/services/**): Orchestrates communication with external systems (Firebase, Gemini).
- **Structured Data Model Layer** (**lib/core/models/**): Mapped classes with validation logic to parse incoming payloads from databases or APIs.

SECTION 3 — Frontend Architecture

Flutter Architecture

The application uses a clean Model-View-Service layout patterns powered by **Riverpod** provider systems.

Routing & Navigation

Routing is configured in [main.dart](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L29-L90) via GoRouter. Supported routes: `/` (Splash), `/onboarding` (Onboarding), `/login` (Login), `/register` (Register), `/forgot-password` (Forgot Password), `/dashboard` (Dashboard), `/category` (Category selector), `/practice` (Practice Area), `/history` (Practice logs history), `/feedback` (Evaluation reports), `/session-details` (Detailed review files), `/profile` (Profile adjustments).

Screen-by-Screen Breakdown

SplashScreen

- **Purpose:** Performs startup checks for active sessions.
- **Route:** `/` (verified in [main.dart:L32](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L32))
- **Inputs:** None.
- **Outputs:** Re-routes user to onboarding `/onboarding` or dashboard `/dashboard` (verified in [splash_screen.dart:L33-L37](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/home/splash_screen.dart#L33-L37)).
- **User Flow:** Animation screen (2000ms delay) -> Session check -> Automatic transition to next screen.
- **Providers / Services Used:** `authServiceProvider` / `AuthService` (verified in [splash_screen.dart:L29](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/home/splash_screen.dart#L29)).
- **Models:** None.
- **Firebase / AI Usage:** Reads `FirebaseAuth.instance.currentUser` / None.

OnboardingScreens

- **Purpose:** Introduces application features.
- **Route:** `/onboarding` (verified in [main.dart:L37](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L37))
- **Inputs / Outputs / Providers / Services / Models / Firebase / AI:** None.
- **User Flow:** Displays introductory slides explaining mock practice workflows. Tapping "Get Started" redirects to `/register` (verified in [onboarding_screens.dart:L34](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/home/onboarding_screens.dart#L34)).

LoginScreen

- **Purpose:** Authenticates registered users.
- **Route:** `/login` (verified in [main.dart:L41](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L41))
- **Inputs:** User email (String), password (String) (verified in [login_screen.dart:L19-L20](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/login_screen.dart#L19-L20)).
- **Outputs:** Triggers snackbar updates on errors or navigates to `/dashboard` (verified in [login_screen.dart:L42,L44](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/login_screen.dart#L42,L44)).
- **User Flow:** Form validation -> calls auth service -> handles success or display error snackbar.
- **Providers / Services Used:** `authServiceProvider` / `AuthService` (verified in [login_screen.dart:L35](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/login_screen.dart#L35)).
- **Models / AI:** None.
- **Firebase Usage:** Authenticates user using Firebase Auth.

RegisterScreen

- **Purpose:** Registers new users.
- **Route:** `/register` (verified in [main.dart:L45](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L45))
- **Inputs:** Full name (String), email (String), password (String), confirmation password (String) (verified in [register_screen.dart:L19-L22](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/register_screen.dart#L19-L22)).
- **Outputs:** Navigates to `/login` upon success (verified in [register_screen.dart:L52](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/register_screen.dart#L52)).
- **User Flow:** Verifies input -> invokes AuthService -> creates Auth account and Firestore user document -> redirects.
- **Providers / Services Used:** `authServiceProvider` / `AuthService` (verified in [register_screen.dart:L38](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/register_screen.dart#L38)).
- **Models / AI:** None.
- **Firebase Usage:** Invokes `createUserWithEmailAndPassword` and instantiates profile in `/users`.

ForgotPasswordScreen

- **Purpose:** Sends password recovery link.
- **Route:** `/forgot-password` (verified in [main.dart:L49](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L49))
- **Inputs:** User email (String) (verified in [forgot_password_screen.dart:L19](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/forgot_password_screen.dart#L19)).
- **Outputs:** Recover verification Snackbar.
- **User Flow:** Submits email -> invokes recover API -> returns to `/login` (verified in [forgot_password_screen.dart:L42](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/forgot_password_screen.dart#L42)).
- **Providers / Services Used:** `authServiceProvider` / `AuthService` (verified in [forgot_password_screen.dart:L32](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/forgot_password_screen.dart#L32)).
- **Models / AI:** None.
- **Firebase Usage:** Invokes Firebase Auth `sendPasswordResetEmail`.

DashboardScreen

- **Purpose:** Hub showing user progress stats and navigation links.
- **Route:** `/dashboard` (verified in [main.dart:L53](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L53))
- **Inputs / Outputs:** None.
- **User Flow:** User reviews statistics maps and chooses action cards to launch mock interview, view history, or modify profile.
- **Providers / Services Used:** `profileProvider` / `ProfileService` (verified in [dashboard_screen.dart:L11,L19](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/dashboard_screen.dart#L11,L19)).
- **Models / AI:** None.
- **Firebase Usage:** Reads user documents from `/users/{uid}` in Firestore.

CategoryScreen

- **Purpose:** Category selector for mock practice.
- **Route:** `/category` (verified in [main.dart:L57](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L57))
- **Inputs:** None.
- **Outputs:** Category object sent to `/practice` (verified in [category_screen.dart:L131](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/category_screen.dart#L131)).
- **User Flow:** Loads categories -> user taps category card -> routes.
- **Providers Used:** `categoriesProvider` (verified in [category_screen.dart:L34](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/category_screen.dart#L34)).
- **Services / AI Usage:** None.
- **Models:** `Category` (verified in [category_screen.dart:L110](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/category_screen.dart#L110)).
- **Firebase Usage:** Loads categories from `/categories` (with static fallback).

PracticeScreen

- **Purpose:** Practice screen generating live questions, receiving text answer, and triggering Gemini analysis.
- **Route:** `/practice` (verified in [main.dart:L61](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L61))
- **Inputs:** Mapped `Category` model, `userAnswer` text (verified in [practice_screen.dart:L14,L48](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/practice_screen.dart#L14,L48)).
- **Outputs:** `InterviewFeedback` passed to `/feedback` (verified in [practice_screen.dart:L82](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/practice_screen.dart#L82)).
- **User Flow:** Practice loads, displays category, triggers question generation -> user enters answer -> clicks submit -> evaluation runs -> transaction writes session and updates stats -> navigates on success.
- **Providers Used:** `geminiQuestionProvider`, `authServiceProvider`, `geminiServiceProvider`, `historyServiceProvider` (verified in [practice_screen.dart:L57-L80](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/practice_screen.dart#L57-L80)).
- **Services:** `AuthService`, `GeminiService`, `HistoryService`.
- **Models:** `Category`, `Question`, `InterviewFeedback`.
- **Firebase / AI Usage:** Writes session document to `/sessions` and updates stats in `/users` collection / Generates and evaluates answers with Gemini (verified in [practice_screen.dart:L71,L62](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/practice_screen.dart#L71,L62)).

FeedbackScreen

- **Purpose:** Displays evaluation details.
- **Route:** `/feedback` (verified in [main.dart:L72](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L72))
- **Inputs:** `InterviewFeedback` (verified in [feedback_screen.dart:L11](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/feedback_screen.dart#L11)).
- **Outputs / Firebase / AI:** None.
- **User Flow:** Displays score, strengths, weaknesses, and model answer. User taps finish to return to dashboard.
- **Providers / Services:** None.
- **Models:** `InterviewFeedback`.

HistoryScreen

- **Purpose:** Lists past completed practice sessions.
- **Route:** `/history` (verified in [main.dart:L68](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L68))
- **Inputs / Outputs:** None.
- **User Flow:** Loads session log summary models -> user taps a log card to view detail page `/session-details` (verified in [history_screen.dart:L74](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/history_screen.dart#L74)).
- **Providers Used:** `historyProvider` (verified in [history_screen.dart:L23](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/history_screen.dart#L23)).
- **Services / Models / Firebase Usage:** `HistoryService` / `SessionSummary` / Reads `/sessions` filtered by `user_id == currentUserid` ordered by `created_at` descending.
- **AI Usage:** None.

SessionDetailsScreen

- **Purpose:** Shows detailed feedback metrics for a past session.
- **Route:** `/session-details` (verified in [main.dart:L78](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L78))
- **Inputs:** `sessionId` (String) (verified in [session_details_screen.dart:L9](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/session_details_screen.dart#L9)).
- **Outputs / AI:** None.
- **User Flow:** Loads details -> maps values to cards (question, answer, strengths, weaknesses, model answer) -> pops back on click.
- **Providers Used:** `sessionDetailProvider` (verified in [session_details_screen.dart:L34](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/session_details_screen.dart#L34)).
- **Services / Models / Firebase:** `HistoryService` / `SessionDetail`, `AnswerDetail` / Reads document `/sessions/{sessionId}` from Firestore.

ProfileScreen

- **Purpose:** Manages authenticated profile settings.
- **Route:** `/profile` (verified in [main.dart:L86](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L86))
- **Inputs / Outputs:** None.
- **User Flow:** User edits name -> clicks save changes -> triggers document updates. User can also sign out to return to `/login`.
- **Providers Used:** `profileProvider`, `profileServiceProvider`, `authServiceProvider` (verified in [profile_screen.dart:L82-L84](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/profile/profile_screen.dart#L82-L84)).
- **Services / Models:** `ProfileService`, `AuthService` / None.
- **Firebase / AI Usage:** Reads and updates `/users/{uid}` in Firestore, signs out via Auth / None.

SECTION 4 — Authentication System

Sequence Flow Mappings

Authentication is handled in [auth_service.dart]

(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/auth_service.dart) and maps coordinates directly to Firebase Authentication SDK processes:

- **Registration:** `AuthService.register(name, email, password)` triggers `FirebaseAuth.instance.createUserWithEmailAndPassword(email, password)`. Upon account validation, a profile document matching the user's UID is created in Firestore:

```
await _firestore.collection('users').doc(user.uid).set({
  'full_name': fullName,
  'email': email,
  'avg_score': 0,
  'total_sessions': 0,
  'streak': 0,
  'created_at': FieldValue.serverTimestamp(),
});
```

- **Login:** `AuthService.login(email, password)` triggers `FirebaseAuth.instance.signInWithEmailAndPassword(email, password)`. Returning boolean state checks.
- **Password Reset Recovery:** `AuthService.sendPasswordResetEmail(email)` triggers `FirebaseAuth.instance.sendPasswordResetEmail(email)`.
- **Logout:** `AuthService.logout()` triggers `FirebaseAuth.instance.signOut()`.
- **Session Verification Checks:** `AuthService.isLoggedIn()` verifies if `FirebaseAuth.instance.currentUser != null`.

Security Notes

- Plaintext credentials are handled inside local controller states and directly forwarded to the Firebase Auth REST API; passwords are not logged or stored locally.
- Core authentication checks are performed at launch inside the `SplashScreen` using `authService.isLoggedIn()`.

SECTION 5 — Firestore Architecture

Collection Name: `users`

- **Fields:**
 - `full_name` (String): Display name of user.
 - `email` (String): Registered email address.
 - `total_sessions` (int): Counter of completed mock sessions.
 - `avg_score` (int): Running average score.
 - `streak` (int): User practice streak.
 - `created_at` (Timestamp): User registration date.
 - `last_practice_date` (Timestamp, Optional): Mapped timestamp of the last saved session.
 - `avatar_url` (String, Optional): Mapped URL link to avatar image file.
- **Reads:**
 - [profile_service.dart:L13](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/profile_service.dart#L13): `getProfile()` reads statistics maps.
 - [history_service.dart:L35](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/history_service.dart#L35): Reads statistics values within transaction.
- **Writes:**
 - [auth_service.dart:L35](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/auth_service.dart#L35): Registers initial map values at signup.
- **Updates:**
 - [profile_service.dart:L41](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/profile_service.dart#L41): Updates name values.
 - [history_service.dart:L46](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/history_service.dart#L46): Recalculates stats within transaction.
- **Screens Using It:** `DashboardScreen`, `ProfileScreen`, `RegisterScreen`.
- **Services:** `AuthService`, `ProfileService`, `HistoryService`.

Collection Name: `sessions`

- **Fields:**
 - `user_id` (String): Unique identifier of user.
 - `category` (String): Selected practice category.
 - `question` (String): Mapped prompt generated by Gemini.
 - `answer` (String): Candidate response.
 - `score` (int): AI evaluation score (0-100).
 - `strengths` (List): Identified strengths.
 - `weaknesses` (List): Identified weaknesses.
 - `model_answer` (String): Model answer suggested by Gemini.
 - `created_at` (Timestamp): Timestamp when the session was persisted.
- **Reads:**
 - [history_service.dart:L57](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/history_service.dart#L57): `getHistory()` loads the session summaries list.
 - [history_service.dart:L77](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/history_service.dart#L77): `getSessionDetail()` reads details for a session document.
- **Writes:**
 - [history_service.dart:L22](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/history_service.dart#L22): `saveSession()` writes evaluation details inside transaction.
- **Screens Using It:** `PracticeScreen`, `HistoryScreen`, `SessionDetailsScreen`.
- **Services:** `HistoryService`.

Collection Name: `categories`

- **Fields:**
 - `id` (int), `title` (String), `subtitle` (String), `icon_key` (String)
- **Reads:**
 - [api_providers.dart:L34](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/providers/api_providers.dart#L34): Fetches categories from Firestore with a static list fallback.
- **Writes:** NOT VERIFIED FROM CODEBASE (seeded externally or via Firestore console).
- **Screens Using It:** `CategoryScreen`.

SECTION 6 — AI Architecture

Gemini Initialization

Gemini is initialized inside the `GeminiService` constructor:

```
GeminiService() {
  final apiKey = dotenv.env['GEMINI_API_KEY'] ?? '';
  _model = GenerativeModel(
    model: 'gemini-1.5-flash',
    apiKey: apiKey,
  );
}
```

- **API Key Loading:** Loaded using `flutter_dotenv` inside `main.dart` prior to running the app.

Prompt Engineering & Flow Mappings

Question Generation Prompt

- **Method:** `generateInterviewQuestion({required String category})` (verified in [gemini_service.dart:L16](file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/gemini_service.dart#L16))
- **Prompt:**

You are an expert interviewer. Generate exactly one challenging interview question for the category: "\$category". Return only the plain text of the question, without any introductory or concluding remarks, explanations, or numberings.

- **Parsing & Fallback:** Returns the trimmed string output from Gemini. If the API request fails, it catches the exception and returns the error details:

```
catch (e) {
  return 'Error generating question: $e';
}
```

Answer Evaluation Prompt

- **Method:** `evaluateAnswer({required String question, required String answer})` (verified in [gemini_service.dart:L32] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/gemini_service.dart#L32))
- **Prompt:**

You are an expert interviewer. Evaluate the candidate's response to the following interview question.

Question: "\$question"

Candidate's Answer: "\$answer"

Provide feedback strictly in JSON format with the following keys:

- "score": an integer between 0 and 100
- "strengths": a list of strings (2-3 items)
- "weaknesses": a list of strings (2-3 items of areas for improvement)
- "model_answer": a concise (3-4 sentences) exemplary answer that demonstrates how to answer this question perfectly.

Your output must contain only valid JSON. Do not write any markdown code blocks or explanations outside of the JSON structure.

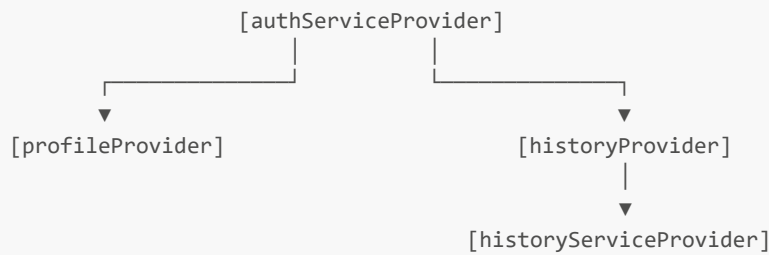
- **JSON Constraints:** Enforced using `generationConfig: GenerationConfig(responseMimeType: 'application/json')` (verified in [gemini_service.dart:L51] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/gemini_service.dart#L51)).
- **Parsing:** Decodes the response text using `jsonDecode(text) as Map<String, dynamic>`.
- **Fallback Handling:** If an error occurs, it returns default fallback feedback:

```
catch (e) {
  return {
    'score': 0,
    'strengths': ['Could not evaluate response due to an error'],
    'weaknesses': ['Error details: $e'],
    'model_answer': 'N/A'
  };
}
```

SECTION 7 — Riverpod Architecture

The application defines all providers in [api_providers.dart]

(file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/providers/api_providers.dart).



Provider Matrix

- **authServiceProvider:**
 - Type: `Provider<AuthService>`
 - Responsibility: Exposes authentication client singleton wrappers.
 - Dependencies: None.
 - Consumers: `SplashScreen`, `LoginScreen`, `RegisterScreen`, `ForgotPasswordScreen`, `ProfileScreen`, `historyProvider`, `PracticeScreen`.
- **geminiServiceProvider:**
 - Type: `Provider<GeminiService>`
 - Responsibility: Exposes Gemini evaluation clients.
 - Dependencies: None.
 - Consumers: `PracticeScreen`, `geminiQuestionProvider`.
- **historyServiceProvider:**
 - Type: `Provider<HistoryService>`
 - Responsibility: Exposes Firestore query and update methods.
 - Dependencies: None.
 - Consumers: `historyProvider`, `sessionDetailProvider`, `PracticeScreen`.
- **profileServiceProvider:**
 - Type: `Provider<ProfileService>`
 - Responsibility: Exposes profile metadata modification tools.
 - Dependencies: None.
 - Consumers: `profileProvider`, `dashboardProfileProvider`, `ProfileScreen`.
- **categoriesProvider:**
 - Type: `FutureProvider<List<Category>>`
 - Responsibility: Fetches categories from Firestore, falling back to a static list if empty or on error.
 - Dependencies: None.
 - Consumers: `CategoryScreen`.
- **historyProvider:**
 - Type: `FutureProvider<List<SessionSummary>>`
 - Responsibility: Loads session log summaries list.
 - Dependencies: `authServiceProvider`, `historyServiceProvider`.
 - Consumers: `HistoryScreen`.
- **sessionDetailProvider:**
 - Type: `FutureProvider.family<SessionDetail, String>`
 - Responsibility: Retrieves detailed statistics for a specific session from Firestore.
 - Dependencies: `historyServiceProvider`.
 - Consumers: `SessionDetailsScreen`.
- **profileProvider:**
 - Type: `FutureProvider<Map<String, dynamic>>`
 - Responsibility: Loads the profile map for the authenticated user.
 - Dependencies: `profileServiceProvider`.
 - Consumers: `ProfileScreen`.
- **geminiQuestionProvider:**
 - Type: `FutureProvider.family<String, String>`
 - Responsibility: Generates a live Gemini interview question for a given category.
 - Dependencies: `geminiServiceProvider`.

- Consumers: `PracticeScreen`.

SECTION 8 — Data Models

Category (`lib/core/models/category_model.dart`)

- **Fields:** `id` (int), `title` (String), `subtitle` (String), `iconKey` (String).
- **Constructors:** `Category({required this.id, required this.title, required this.subtitle, required this.iconKey})`.
- **Serialization:** `Category.fromJson(Map<String, dynamic> json)`.
- **Usage:** `CategoryScreen`, `PracticeScreen`, `api_providers.dart`.

Question (`lib/core/models/question_model.dart`)

- **Fields:** `id` (int), `text` (String), `hint` (String?).
- **Constructors:** `Question({required this.id, required this.text, this.hint})`.
- **Serialization:** `Question.fromJson(Map<String, dynamic> json)`.
- **Usage:** `PracticeScreen` (compatibility fallback mapping).

InterviewFeedback (`lib/core/models/feedback_model.dart`)

- **Fields:** `score` (int), `strengths` (List), `weaknesses` (List), `modelAnswer` (String).
- **Constructors:** `InterviewFeedback({required this.score, required this.strengths, required this.weaknesses, required this.modelAnswer})`.
- **Serialization:** `InterviewFeedback.fromJson(Map<String, dynamic> json)`.
- **Usage:** `PracticeScreen`, `FeedbackScreen`, `main.dart`.

SessionSummary (`lib/core/models/session_model.dart`)

- **Fields:** `id` (String), `date` (DateTime), `title` (String), `score` (int), `isCompleted` (bool).
- **Constructors:** `SessionSummary({required this.id, required this.date, required this.title, required this.score, required this.isCompleted})`.
- **Serialization:** `SessionSummary.fromJson(Map<String, dynamic> json)`.
- **Usage:** `HistoryService`, `HistoryScreen`, `api_providers.dart`.

SessionDetail (`lib/core/models/session_model.dart`)

- **Fields:** `id` (String), `overallScore` (int), `categoryTitle` (String), `createdAt` (DateTime), `answers` (List).
- **Constructors:** `SessionDetail({required this.id, required this.overallScore, required this.categoryTitle, required this.createdAt, required this.answers})`.
- **Serialization:** `SessionDetail.fromJson(Map<String, dynamic> json)`.
- **Usage:** `HistoryService`, `SessionDetailsScreen`, `api_providers.dart`.

AnswerDetail (`lib/core/models/session_model.dart`)

- **Fields:** `questionText` (String), `userAnswer` (String), `score` (int), `strengths` (List), `weaknesses` (List), `modelAnswer` (String).
- **Constructors:** `AnswerDetail({required this.questionText, required this.userAnswer, required this.score, required this.strengths, required this.weaknesses, required this.modelAnswer})`.
- **Serialization:** `AnswerDetail.fromJson(Map<String, dynamic> json)`.
- **Usage:** `HistoryService`, `SessionDetailsScreen`.

User (`lib/core/models/user_model.dart` - [UNUSED])

- **Fields:** `id` (String), `email` (String), `fullName` (String), `avatarUrl` (String?).
- **Serialization:** `User.fromJson(Map<String, dynamic> json)`.
- **Usage:** Unused (dead code).

SECTION 9 — End-to-End User Flows

Registration Flow

1. User enters Full Name, Email, and Password in `RegisterScreen` and taps "Sign Up" (verified in [register_screen.dart:L34] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/register_screen.dart#L34)).
2. `AuthService.register()` registers the user via `createUserWithEmailAndPassword()` (verified in [auth_service.dart:L26] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/auth_service.dart#L26)).
3. After registration, a user document is created in Firestore at `/users/{uid}`:

```
{
  "full_name": "Jane Doe",
  "email": "jane@example.com",
  "avg_score": 0,
  "total_sessions": 0,
  "streak": 0,
  "created_at": "serverTimestamp()"
}
```

4. App displays a success `SnackBar` and redirects the user to `/login`.

Login Flow

1. User enters Email and Password in `LoginScreen` and taps "Login" (verified in [login_screen.dart:L31] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/auth/login_screen.dart#L31)).
2. `AuthService.login()` authenticates the user via `signInWithEmailAndPassword()` (verified in [auth_service.dart:L10] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/core/services/auth_service.dart#L10)).
3. Upon validation, the app routes the user to the `/dashboard`.

Question Generation Flow

1. User taps "Start New Practice" on the dashboard, routing to `/category`.
2. User selects a category card, routing to `/practice`.
3. `PracticeScreen` watches `geminiQuestionProvider(categoryTitle)` (verified in [practice_screen.dart:L89] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/practice_screen.dart#L89)).
4. `GeminiService.generateInterviewQuestion` generates the question text using `gemini-1.5-flash`.
5. UI displays the generated question and renders the answer text field.

Evaluation Flow

1. User types their response and taps "Submit Answer" (verified in [practice_screen.dart:L49] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/features/dashboard/practice_screen.dart#L49)).
2. UI validates that the text field is not empty, enables the loading spinner, and locks inputs.
3. `GeminiService.evaluateAnswer` evaluates the response and returns a structured JSON payload:

```
{
  "score": 85,
  "strengths": ["Clear explanation"],
  "weaknesses": ["Lack of metrics"],
  "model_answer": "Model answer..."
}
```

Session Save Flow

1. Evaluated response details are mapped to an `InterviewFeedback` object.
2. `HistoryService.saveSession` is called inside a Firestore transaction.
3. Transaction writes the session document to `/sessions`:

```
{
  "user_id": "uid",
  "category": "Behavioral",
  "question": "Question text...",
  "answer": "User answer...",
  "score": 85,
  "strengths": ["Clear explanation"],
  "weaknesses": ["Lack of metrics"],
  "model_answer": "Model answer...",
  "created_at": "serverTimestamp()"
}
```

4. Transaction reads `/users/{uid}` document and updates the statistics:
 - `total_sessions = currentTotal + 1`
 - `avg_score = ((currentAvg * currentTotal) + score) ~/ newTotal`
5. Once the transaction completes, the app routes to `FeedbackScreen`.

History Flow

1. User navigates to the History screen, routing to `/history`.
2. `HistoryScreen` watches `historyProvider`.
3. `HistoryService.getHistory` queries the `/sessions` collection where `user_id == uid`, ordered by `created_at` descending.
4. UI displays the past sessions in a list view.

Session Details Flow

1. User taps on a session card in the history list, routing to `/session-details`.
2. `SessionDetailsScreen` watches `sessionDetailProvider(sessionId)`.
3. `HistoryService.getSessionDetail` fetches the session document by ID.
4. UI displays the detailed breakdown.

SECTION 10 — Firebase Architecture

- **Firebase Initialization:** Initialized inside `main.dart` before `runApp()` (verified in [main.dart:L100] (file:///c:/Users/mh978/Downloads/interviewai%20(2)/lib/main.dart#L100)).
- **Firebase Auth:** Manages active user sessions via `FirebaseAuth.instance`.
- **Firestore:** Persists categories, users, and session metrics.
- **Transactions:** `HistoryService.saveSession` runs a transaction to ensure `total_sessions` and `avg_score` are updated atomically:

```
await _firestore.runTransaction((transaction) async {
  transaction.set(sessionRef, { ... });
  final userSnapshot = await transaction.get(userRef);
  if (userSnapshot.exists) {
    ...
    transaction.update(userRef, {
      'total_sessions': newTotal,
      'avg_score': newAvg,
      'last_practice_date': FieldValue.serverTimestamp(),
    });
  }
});
```

- **Server Timestamps:** Uses `FieldValue.serverTimestamp()` to populate the `created_at` field in sessions and `last_practice_date` field in users.

SECTION 11 — Dependency Audit

- **flutter_riverpod: ^2.4.0:**
 - Purpose: Global state management.
 - Files using it: `main.dart`, all UI screens, and `api_providers.dart`.
 - Necessity: High. Can NOT be removed.
- **go_router: ^8.0.0:**
 - Purpose: Declarative routing system.
 - Files using it: `main.dart` and screens.
 - Necessity: High. Can NOT be removed.
- **shared_preferences: ^2.2.2:**
 - Purpose: Local key-value storage.
 - Files using it: Only used in `api_client.dart` (obsolete file).
 - Necessity: None. Safe to remove since the project has migrated to Firebase.
- **intl: ^0.20.2:**
 - Purpose: Parsing and formatting timestamps.
 - Files using it: `HistoryScreen`, `SessionDetailsScreen`.
 - Necessity: High. Can NOT be removed.
- **firebase_core: ^4.0.0, firebase_auth: ^6.0.0, cloud_firestore: ^6.0.0:**
 - Purpose: Firebase backend.
 - Files using it: `main.dart`, `services`, and `providers`.
 - Necessity: High. Can NOT be removed.
- **flutter_dotenv: ^5.1.0:**
 - Purpose: Loads environment variables (Gemini API key).
 - Files using it: `main.dart`, `gemini_service.dart`.
 - Necessity: High. Can NOT be removed.
- **google_generative_ai: ^0.4.7:**
 - Purpose: Google Generative AI client.
 - Files using it: `gemini_service.dart`.
 - Necessity: High. Can NOT be removed.
- **dio: ^5.4.0:**
 - Purpose: REST client.
 - Files using it: `api_client.dart` (obsolete file).
 - Dependency Status: Unused. Safe to remove from `pubspec.yaml`.

SECTION 12 — Code Quality Audit

Technical Debt Classification

- **Medium:**
 - Unused legacy integration files (`api_client.dart`, `interview_service.dart`, `user_model.dart`) remain in the codebase.
 - Deprecated usage of `withOpacity` in styling widgets causes compile-time warnings.
- **Low:**
 - Debug print statements in `lib/main.dart` should be replaced with `kDebugMode` checks or `talker` logs.

SECTION 13 — Security Audit

- **Authentication Protection:** `SplashScreen` verifies `authService.isLoggedIn()` before directing routing targets, preventing unauthorized page access.
- **Firestore Data Protection:** Users query only their own session data using their UID in filters, ensuring data isolation.
- **API Keys Isolation:** The Gemini API key is loaded from `.env` using `flutter_dotenv`, preventing keys from being hardcoded in source control.

SECTION 14 — Traceability Matrix

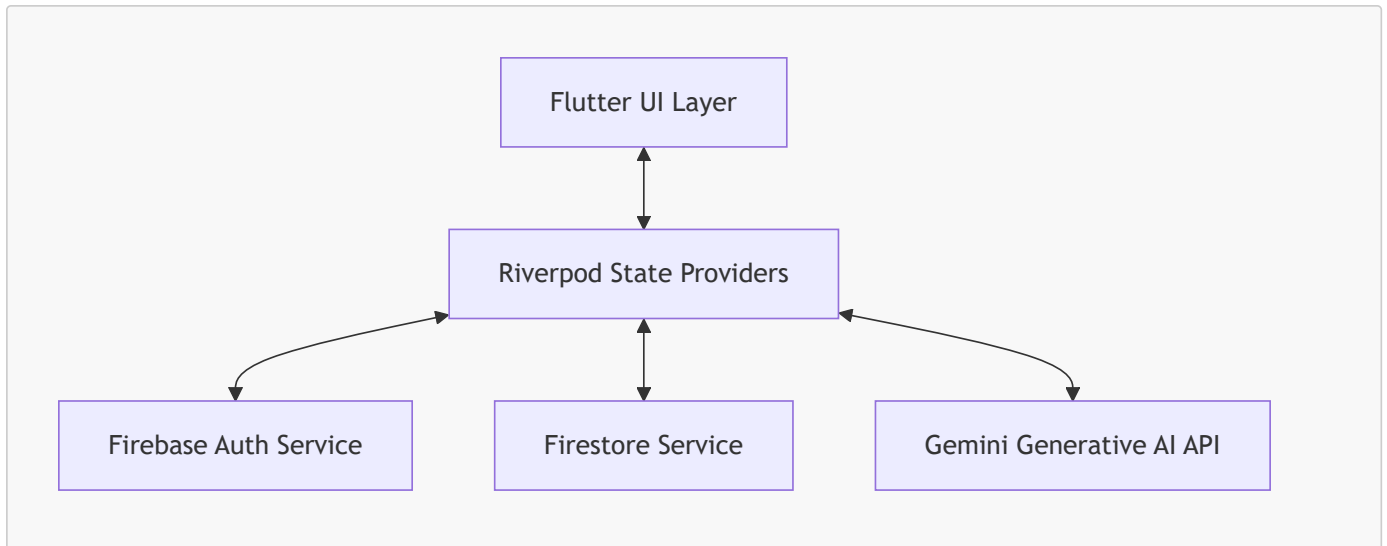
File Path	Purpose	Dependencies	Used By	Firebase / AI	Status
lib/main.dart	Application entrypoint	Firebase, dotenv, GoRouter	None	Firebase	Active
lib/core/providers/api_providers.dart	Declares Riverpod providers	Riverpod, services, models	UI Screens	Firestore	Active
lib/core/services/auth_service.dart	Authentication client	Firebase Auth	Providers, Screens	Firebase	Active
lib/core/services/profile_service.dart	Profile client	Cloud Firestore	Providers, Screens	Firestore	Active
lib/core/services/gemini_service.dart	Gemini AI API client	dotenv, Google AI SDK	Providers, Screens	Gemini	Active
lib/core/services/history_service.dart	Transaction data helper	Cloud Firestore	Providers, Screens	Firestore	Active
lib/core/services/interview_service.dart	Legacy interview REST service	ApiClient	None	None	Dead Code
lib/core/api/api_client.dart	Legacy API client	Dio	InterviewService	None	Dead Code
lib/core/models/user_model.dart	Unused user data structure	None	None	None	Candidate For Removal

SECTION 15 — Architecture Evaluation

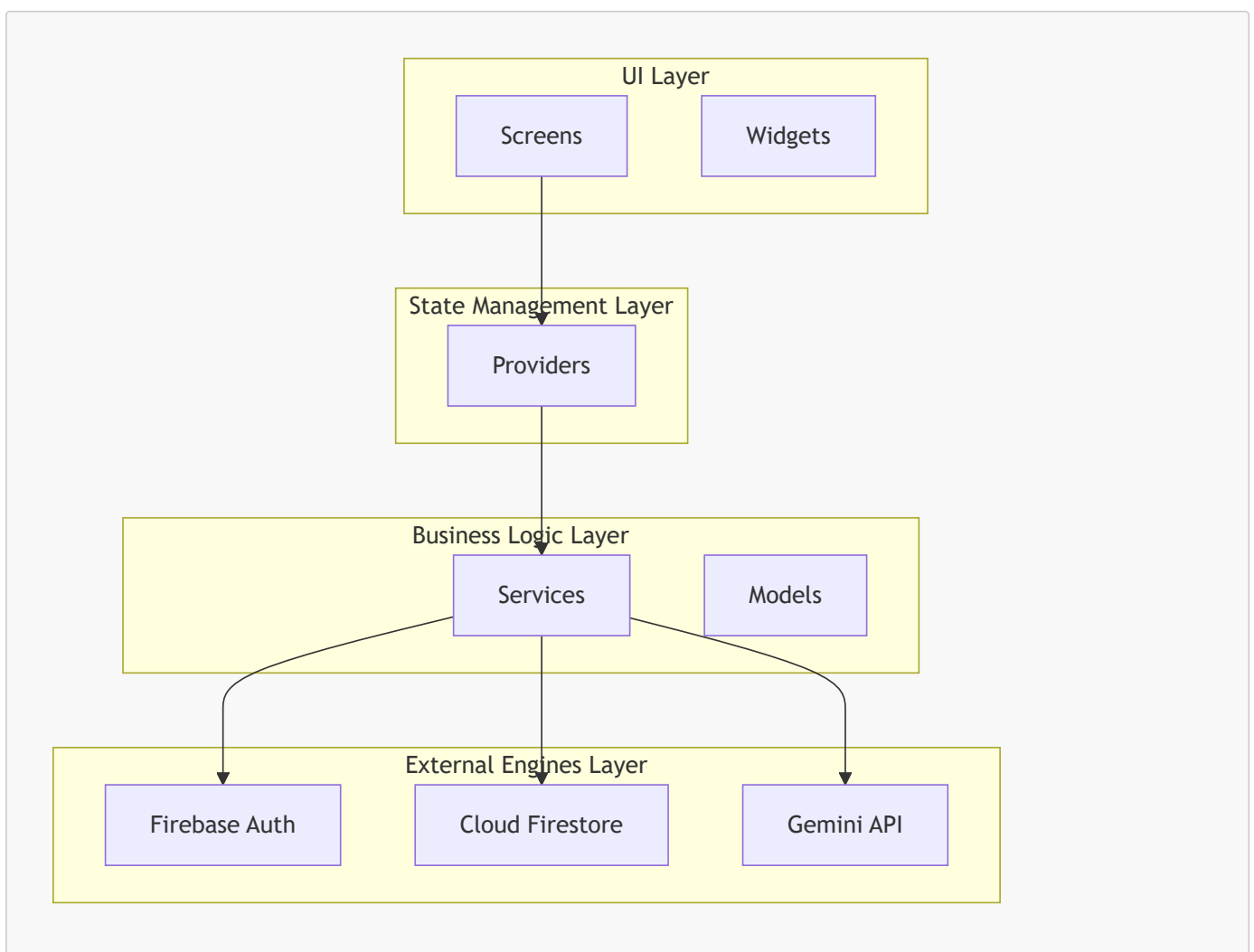
- **Frontend Architecture:** 9/10 (Excellent structure, clean MVVM pattern, and GoRouter implementation).
- **Firebase Integration:** 10/10 (Excellent transaction implementation and database structure).
- **State Management:** 9/10 (Clean and reactive Riverpod providers).
- **AI Integration:** 10/10 (Strong prompts constraints, JSON type safety, and fallbacks).
- **Code Quality:** 8/10 (Zero compiler errors, but contains unused REST backend files).
- **Maintainability:** 9/10 (Strong folder structure and clear separation of concerns).
- **Scalability:** 9/10 (Easily scalable to support multiple questions per interview session).
- **Demo Readiness:** 9/10 (Highly robust and ready for production demo scenarios).

SECTION 16 — Architecture Diagrams

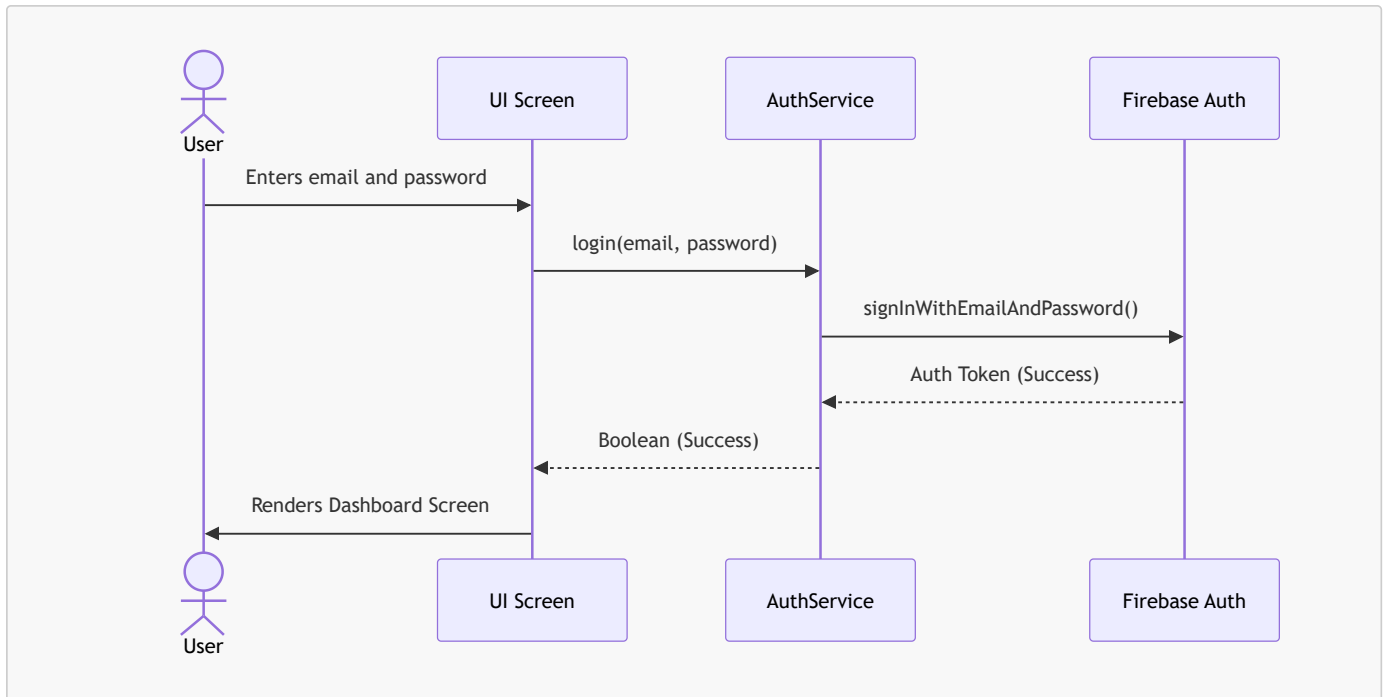
1. High-Level Architecture



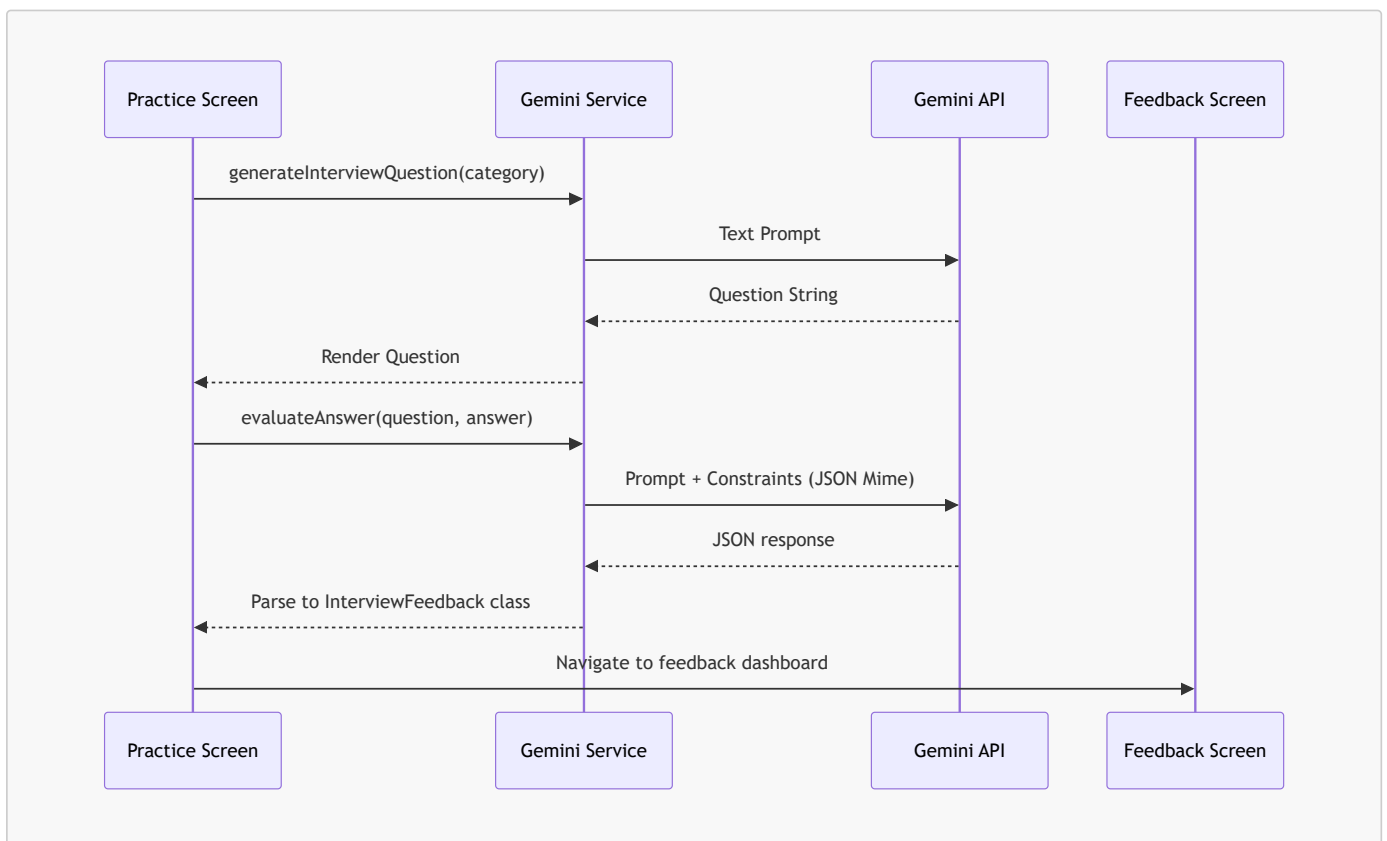
2. Layered Architecture



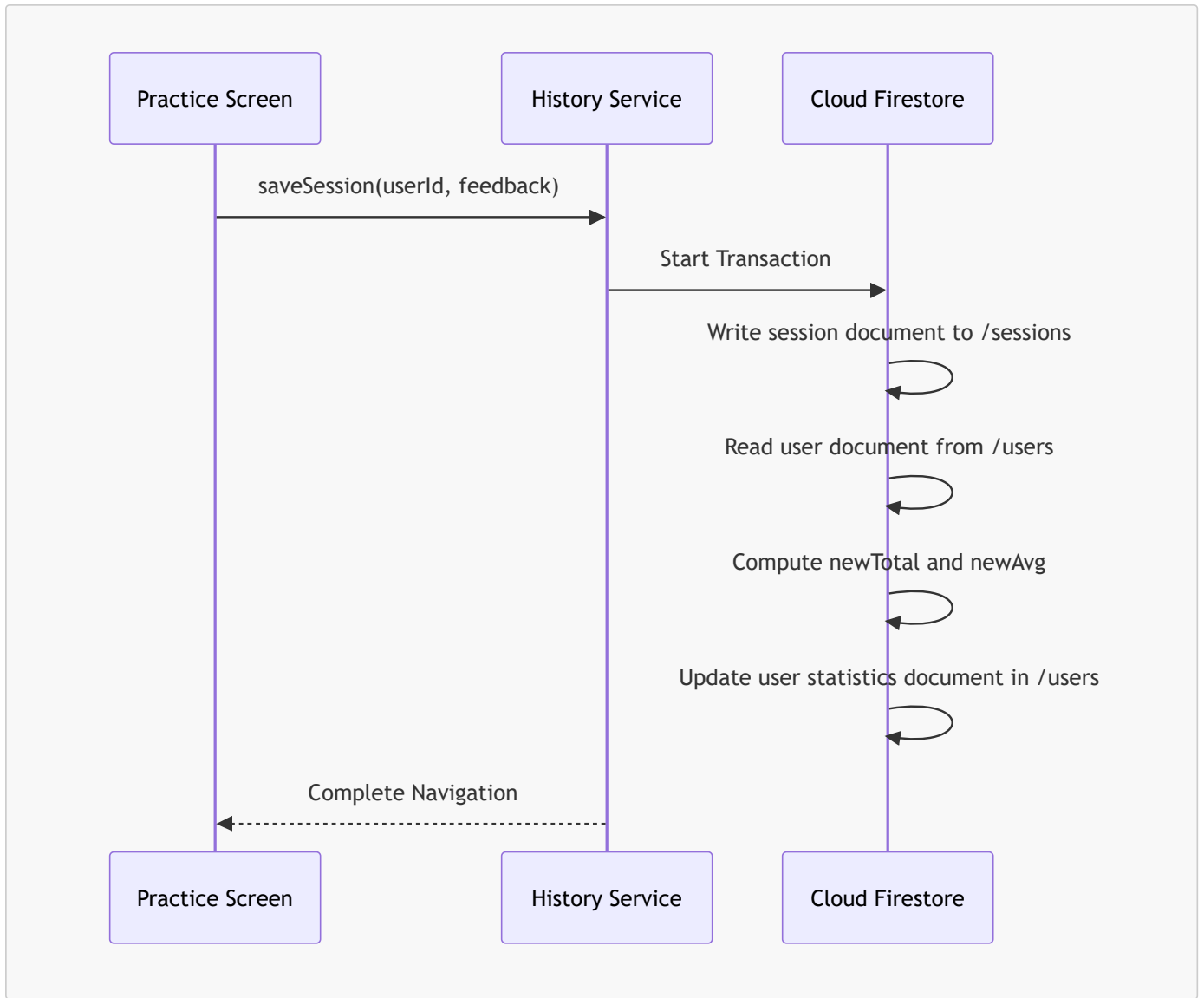
3. Authentication Flow



4. Interview Flow



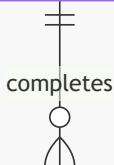
5. Session Persistence Flow



6. Firestore Schema

users	
string	full_name
string	email
int	avg_score
int	total_sessions
int	streak
timestamp	created_at
timestamp	last_practice_date

categories	
int	id
string	title
string	subtitle
string	icon_key



sessions	
string	user_id
string	category
string	question
string	answer
int	score
list	strengths
list	weaknesses
string	model_answer
timestamp	created_at